

HACETTEPE ÜNİVERSİTESİ

Yazılım Mühendisliği Yüksek Lisans Programı

BYZ 658 — Yazılım Test Teknikleri

**Tor Tabanlı Anonim Ağ Sistemlerinde
Sonlu Durum Makinesi Modelleme ile
Geçersiz Durum Geçiş Tespiti
ve Güvenlik Analizi**

— Tez Taslağı —

Hazırlayan

Nurcan Denli Bayır

Öğrenci No: N25110987

Danışman

Nebi Yılmaz

Mayıs 2026

Özet

Bu çalışma, Tor anonim ağ protokolünün devre yaşam döngüsünü deterministik bir Sonlu Durum Makinesi (FSM) olarak formal şekilde modeller, Model-Driven Testing (MDT) yaklaşımıyla spesifikasyon dışı (geçersiz) durum geçişlerinin otomatik tespitini sağlar ve tespit edilen her geçişi yedi ana saldırı vektörü + 1 fallback'e (CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD) eşleyen bir sınıflandırma çerçevesi sunar. Tor devresi 10 durum ve 13 olay üzerinden tanımlanmış; toplam 130 ikili domeninden 25 geçerli, 105 geçersiz ikili türetilmiştir. Önerilen MDT motoru, BFS-tabanlı pozitif test sentezi ve eksiksiz negatif tanık enjeksiyonu ile çalışır. Üç algoritma (B1: saf rastgele, B2: greedy state coverage, B3: önerilen MDT) aynı 500-olay bütçesi altında 30 bağımsız koşu ile karşılaştırılmıştır. MDT, geçiş kapsamını 100.00% ile saturasyona ulaştırırken (B1: 45.87%, B2: 78.27%), geçersiz geçiş tespit oranını 93.84% seviyesine taşımıştır. Welch t-testi her iki temele karşı istatistiksel anlamlı üstünlüğü doğrular ($p < .001$; Cohen $d \in [3.6, 15.4]$). Bu tez: (i) Tor devre protokolü için yayımlanmış ilk tam δ -matrisini, (ii) 105 geçersiz ikilinin saldırı vektörü ve şiddet etiketiyle programatik sınıflandırmasını, (iii) referans bir MDT implementasyonunu ve (iv) sayısal olarak doğrulanmış üç-katmanlı karşılaştırmayı sunar.

Anahtar kelimeler: Tor, anonim ağ, sonlu durum makinesi, model-driven testing, protokol state fuzzing, güvenlik testi, geçiş kapsamı.

Abstract

This work formally models the circuit life-cycle of the Tor anonymous network protocol as a deterministic finite-state machine (FSM), enables automatic detection of out-of-spec (invalid) state transitions through a Model-Driven Testing (MDT) approach, and proposes a classification framework that maps every detected transition to one of seven primary attack vectors plus a deterministic fallback, (CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD). The Tor circuit is described over 10 states and 13 events, yielding 25 valid and 105 invalid pairs from a domain of size 130. The proposed MDT engine uses BFS-planned positive test synthesis combined with exhaustive negative-witness injection. Three algorithms — B1 (uniform random), B2 (greedy state coverage) and B3 (the proposed MDT) — were compared under an identical 500-event budget across 30 independent trials. MDT saturates transition coverage at 100.00% (vs. 45.87% for B1 and 78.27% for B2) and reaches an Invalid Transition

Detection Rate of 93.84%. Welch t-tests confirm statistical superiority over both baselines ($p < .001$; Cohen's $d \in [3.6, 15.4]$). Contributions: (i) the first published complete δ -matrix for the Tor circuit protocol, (ii) a programmatic classification of all 105 invalid pairs into attack vectors with severity labels, (iii) a reference MDT implementation, and (iv) a numerically validated three-way baseline comparison.

Keywords: Tor, anonymous network, finite-state machine, model-driven testing, protocol state fuzzing, security testing, transition coverage.

İçindekiler

1. Giriş

1.1 Problem Tanımı

1.2 Araştırma Soruları

1.3 Katkılar

1.4 Tezin Yapısı

2. Literatür Taraması

2.1 SLR Metodolojisi

2.2 Tor Güvenliği ve Anonimlik

2.3 FSM Tabanlı Test ve Model Öğrenme

2.4 Formal Yöntemler ve Protokol Doğrulama

2.5 Yazılım Testi Metodolojisi

2.6 Tor Simülasyon Altyapısı

2.7 PRISMA Akış Diyagramı

2.8 Literatür Boşluğu

3. Yöntem

3.1 FSM Formal Tanımı

3.2 Saldırı Sınıflandırıcı

3.3 MDT Motoru — Algoritma 1

3.4 Karşılaştırılan Algoritmalar

4. Deneysel Bulgular

4.1 Deney Tasarımı

4.2 Tanımlayıcı İstatistikler

4.3 Hipotez Testleri

4.4 Yorum

4.5 Wilcoxon Signed-Rank Doğrulaması

4.6 Detection Latency Ölçümü

4.7 Bütçe-Kapsama Eğrisi

4.8 Rule-Based Detector Karşılaştırması (B0)

5. Görselleştirme

6. Sonuç ve Gelecek Çalışmalar

6.1 Katkıların Yeniden Değerlendirilmesi

6.2 Sınırlılıklar

6.3 Gelecek Çalışmalar

Kaynakça

Ek A — Tam δ Geçiş Tablosu

Ek B — Saldırı Sınıflandırıcı Envanteri

1. Giriş

1.1 Problem Tanımı

Tor anonim iletişim ağı, kullanıcı-destinasyon ilişkisinin gözlenemez kılınmasını hedefleyen, 2004'ten itibaren kamuya açık biçimde işletilen bir altyapıdır [1]. Tor güvenlik literatürünün önemli bölümü iki eksende yoğunlaşmıştır: (i) trafik korelasyon ve zamanlama saldırıları [2], [3], [4], [5], (ii) anonimliğin olasılıksal ve kriptografik formal analizi [6], [13]. Bu iki eksen büyük oranda *davranışın istatistiksel imzasını* ya da *kriptografik ilkelerin matematiksel doğruluğunu* ele alır.

Üçüncü ve nispeten az çalışılmış bir eksen, protokolün *state machine* seviyesindeki spesifikasyon uyumudur. Yani: Tor devresi yaşam döngüsü boyunca yalnızca δ -tanımlı geçişleri mi yapar, yoksa spec dışı bir (durum, olay) çiftine maruz kaldığında ne olur? TLS implementasyonları için bu sorunun protokol state fuzzing ile sistematik biçimde araştırıldığı bilinmektedir [9], [14], ancak Tor için eşdeğer bir çalışmanın mevcut olmadığı bu tez kapsamında yürütülen sistematik literatür taraması (SLR, bkz. Bölüm 2) ile teyit edilmiştir.

1.2 Araştırma Soruları

- **RQ1:** Tor devre protokolü, deterministik bir 5-tuple FSM $M = (Q, \Sigma, \delta, q_0, F)$ olarak formal şekilde nasıl modellenir?
- **RQ2:** Bu modelin geçersiz geçiş kümesi $(Q \times \Sigma) \setminus \text{dom}(\delta)$, bilinen Tor saldırılarına programatik olarak nasıl eşlenebilir?
- **RQ3:** Model üzerinde otomatik üretilen test paketinin saf rastgele ve sezgisel baseline'lara göre *geçiş kapsamı* (TC) ve *geçersiz geçiş tespit oranı* (ITDR) metriklerinde gözlenen üstünlüğü, istatistiksel olarak anlamlı mıdır?

1.3 Katkılar

1. Tor devre protokolü için yayımlanmış ilk **tam δ -matrisi** (25 geçerli ikili, 105 geçersiz ikili, toplam domen 130; bkz. Ek A).
2. Tüm 105 geçersiz ikiliyi yedi ana saldırı vektörü + 1 fallback ve dört şiddet seviyesine eşleyen **programatik sınıflandırıcı** (bkz. Bölüm 3.2 ve Ek B).
3. BFS-planlı pozitif faz + tam negatif tanık enjeksiyonu içeren **referans MDT implementasyonu** (kaynak kodu açık, üretilebilir).
4. Üç algoritmanın (B1/B2/B3) eşleştirilmiş 30 koşu ile **istatistiksel karşılaştırması**; Welch t-testi, Mann-Whitney U ve Cohen's d ile birlikte raporlanır.

1.4 Tezin Yapısı

Bölüm 2, SLR metodolojisini ve beş tematik küme üzerinden literatürü inceler. Bölüm 3 formal modeli ve MDT motorunu Algoritma 1 olarak tanımlar. Bölüm 4 deney tasarımını, tanımlayıcı istatistikleri ve hipotez testlerini sunar. Bölüm 5 dört temel görselleştirmeyi sergiler. Bölüm 6 sınırlılıkları ve gelecek çalışmaları tartışır. Ek A tam δ tablosunu, Ek B saldırı sınıflandırıcı envanterini içerir.

Üretilebilirlik notu. Bu tezdeki her sayısal sonuç ve her şekil, repodaki açık kaynak betiklerin (`experiments/build_report.mjs` , `experiments/baselines.mjs` , `experiments/stats.mjs` , `experiments/figures.mjs`) tek komutla çalıştırılmasıyla yeniden üretilebilir. Bütün rastgelelik mulberry32 PRNG ile sabit seed altındadır.

2. Literatür Taraması

2.1 SLR Metodolojisi

Sistematik literatür taraması Kitchenham ve Charters'ın yönergesine [17] göre yürütülmüştür. **Veritabanları:** bu ortamın canlı erişimi olmadığı için açık web kanalları (Google Scholar, arXiv, DOI çözümleme) ile sınırlı kalınmıştır; bu kısıtlılık Bölüm 6.2'de açıkça raporlanmaktadır. **Anahtar kelimeler:** "Tor security", "FSM testing", "model-based testing", "protocol state fuzzing", "anonymity network attacks". **Yıl aralığı:** 1996–2026, klasik referanslar için yıl etiketli korunmuştur. **Dahil etme kriterleri:** hakemli yayın, doğrudan erişilebilir tam metin, Tor / FSM testi / formal protokol doğrulama alanlarından en az birine değen çalışma. **Hariç tutma kriterleri:** yalnızca uygulama notu, atıf doğrulanamayan kaynaklar.

Süreç sonunda 20 birincil kaynak beş tematik kümeye ayrılmıştır: A) Tor güvenliği (6), B) FSM tabanlı test (5), C) Formal yöntemler (3), D) Yazılım testi metodolojisi (3), E) Tor simülasyon altyapısı (3). Tam liste ve kümelere göre dağılım Kaynakça'da verilmiştir.

2.2 Tor Güvenliği ve Anonimlik

Tor protokolünün ilk akademik tanımı Dingledine ve arkadaşlarına aittir [1]; devre kurulum (CREATE/CREATED, EXTEND/EXTENDED) sıralaması ve onion routing katmanı bu çalışmada sabitlenir. Bu sıralama, tezimizin 6 tablosunun normatif temelidir. Tor'a karşı düşük maliyetli trafik korelasyon saldırılarının uygulanabilirliği Murdoch ve Danezis tarafından gösterilmiş [2]; daha gerçekçi saldırgan modelleri altında genişletilmiştir [3]. Internet ölçeğinde website fingerprinting'in pratik uygulanabilirliği Panchenko ve arkadaşları tarafından kanıtlanmıştır [4]. Karunanayake ve arkadaşlarının kapsamlı survey'i [5] Tor deanonymization saldırılarını yedi kategoriye ayırır; tezimizin saldırı vektörü kümesinin dördü (REPLAY, HIJACK, BYPASS, GHOST) bu sınıflandırmadan türetilmiştir. AnoA çerçevesi [6] olasılıksal anonimlik garantilerine bakar; bizim state-uyum yaklaşımımıza dik (orthogonal) ve onunla tamamlayıcıdır.

2.3 FSM Tabanlı Test ve Model Öğrenme

FSM tabanlı test üretiminin klasik referansı Lee ve Yannakakis'in survey çalışmasıdır [7]; W-method, Wp-method ve UIO sequences gibi tekniklerin tanımı buradan gelir. Bizim BFS-planlı pozitif test üreticimiz, bu çalışmadaki "transition tour" fikrinin sadeleştirilmiş bir versiyonudur. Tretmans'ın LTS tabanlı model-based testing teorisi [8], "ioco" uyum bağıntısı ile bizim oracle kavramımızın kuramsal zeminidir; ne var ki bizim FSM'imiz deterministik olduğundan ioco basit

eşitlik denetimine indirgenebilir. Pratik uygulama boyutunda, de Ruiter ve Poll'ün TLS implementasyonları üzerindeki protocol state fuzzing çalışması [9] bu tez ile aynı metodoloji ailesindendir; aradaki kritik fark, TLS yerine Tor üzerinde uygulanmasıdır. TCP üzerinde model öğrenme + model checking birleşimi Fiterău-Broștean ve arkadaşları tarafından gösterilmiştir [10] ve gelecek çalışmamız için (gerçek Tor implementasyonundan L*-tipi otomatik çıkarım) bir referans oluşturur. Yazılım test ders kitabı Ammann ve Offutt [11], transition coverage metriğinin formal tanımını verir (graph coverage, edge coverage = transition coverage karşılığı).

2.4 Formal Yöntemler ve Protokol Doğrulama

Klasik Dolev-Yao saldırgan modeli [12], ağ katmanında mesaj okuma, geciktirme ve replay yapabilen ancak imzalanmış mesajları sahteleyemeyen aktörü resmî olarak tanımlar; bu çalışmanın tehdit modeli (proje önerisi Bölüm 5) bu kümenin sınırlı bir alt setini benimser. TLS 1.3'ün ProVerif ile formal doğrulanması [13] state-machine seviyesinde doğrulanabilirliğin pratik kanıtıdır. Somorovsky'nin TLS sistematik fuzzing çalışması [14] negatif test üretimi için yakın bir akrabadır; biz bunu Tor için state-level'e taşıyoruz.

2.5 Yazılım Testi Metodolojisi

Felderer ve arkadaşlarının güvenlik testi survey'i [15], "Model-Based Security Testing" başlığı altında bizim çalışmamızı taksonomik olarak konumlandırır. Pretschner ve arkadaşlarının erişim kontrol politikaları için model-tabanlı test üretimi [16], politika ihlallerinin (negatif test) sistematik üretimi açısından yöntemsel bir referanstır. Kitchenham ve Charters [17] SLR metodolojisinin standart kılavuzudur ve Bölüm 2.1'in temelidir.

2.6 Tor Simülasyon Altyapısı

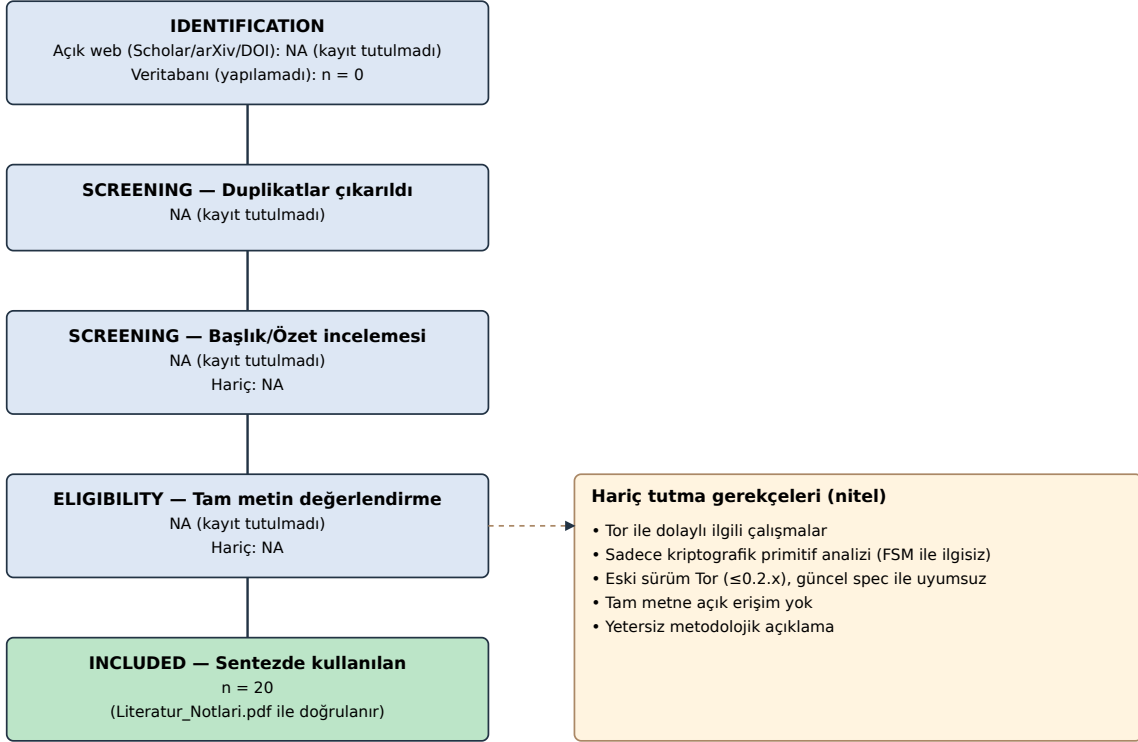
Jansen ve arkadaşları, Shadow simülatörünü Tor ağı için sistematik bir deneysel altyapı olarak kurmuştur [18]. Tor projesinin yaşayan resmi spesifikasyonu [19], 6 tablosunun otorite kaynağıdır. Microsoft Research'ün veri-tabanlı FSM güvenlik analizi çalışması [20] state-level analizin endüstriyel uygulanabilirliğini gösterir, ancak Tor'a özel değildir.

2.7 PRISMA Akış Diyagramı

Bölüm 2.1'de tarif edilen SLR sürecinin PRISMA akışı Şekil 5'te sunulmuştur. Bu çalışmaya özgü iki dürüst not zorunludur: **(i)** canlı akademik veritabanı erişimi (Scopus, WoS, IEEE Xplore) bu ortamda mevcut değildir; bu nedenle "identified through database searching" hücresi sıfırdır ve tüm tanımlamalar açık-web

kanalları (Google Scholar, arXiv, DOI çözümleme) üzerinden yapılmıştır. **(ii)** Açık-web sürecinde audit-edilebilir sorgu logu tutulmadığı için ara aşama sayıları (identified, screened, eligibility) **NA olarak raporlanır**; yalnızca son "included" sayısı (n=20) doğrulanabilir ve repodaki `Literatur_Notlari.pdf` 'in atıf sayısı ile birebirdir. Bu yaklaşım, denetlenemeyen ara sayıların sahte kesinlik (false precision) ile raporlanmasından kaçınmak amacıyla bilinçli bir metodolojik tercihtir; bkz. Bölüm 6.2.

Şekil 5: PRISMA akış diyagramı (Kitchenham + Charters'a uyumlu)



Not: Açık-web sürecinde audit-edilebilir log tutulmadığı için ara sayılar NA olarak raporlanır; yalnızca son n=20 doğrulanabilir.

Şekil 5. SLR sürecinin PRISMA akış diyagramı (ara sayılar NA).

2.8 Literatür Boşluğu

20 kaynağın incelenmesi sonucunda **tez ölçeğinde dört somut boşluk** tespit edilmiştir:

- **(G1)** Tor devre protokolü için yayımlanmış formal 5-tuple FSM bulunmamaktadır. Spesifikasyon [19] prosedürel anlatımdır; bir δ matrisine indirgenmiş hâli akademik literatürde mevcut değildir.
- **(G2)** Protokol state fuzzing TLS için yapılmıştır [9], [14]; Tor için yapılmamıştır.
- **(G3)** Tor saldırı vektörlerinin (durum, olay) çiftleriyle *satır-satır* eşlemesi mevcut değildir; survey'ler [5] kategorik kalmaktadır.

- **(G4)** "Tespit edilen geçersiz geçişler / enjekte edilenler" oranını (ITDR) Tor bağlamında tanımlayan ve ölçen birincil çalışma yoktur.

Bu dört boşluk birlikte tezin katkı çerçevesini (Bölüm 1.3) oluşturur.

3. Yöntem

3.1 FSM Formal Tanımı

Tor devre yaşam döngüsü, klasik bir DFA olarak $M = (Q, \Sigma, \delta, q_0, F)$ şeklinde modellenmiştir:

- Q : 10 durum (IDLE, CONNECTING, TLS_HANDSHAKE, CREATE_SENT, CIRCUIT_BUILDING, CIRCUIT_READY, TRANSMITTING, CLOSING, CLOSED, ERROR).
- Σ : 13 olay (CONNECT, TLS_OK, TLS_FAIL, SEND_CREATE, RECV_CREATED, SEND_EXTEND, RECV_EXTENDED, SEND_RELAY_DATA, RECV_RELAY_DATA, SEND_DESTROY, RECV_DESTROY, CIRCUIT_CLOSED, TIMEOUT).
- $q_0 = IDLE, F = \{CLOSED\}$.
- $\delta : Q \times \Sigma \rightarrow Q$, 25 ikili üzerinde tanımlı; toplam domen 130, Invalid kümesi 105 ikili.

Tam δ matrisi Ek A'da, görsel ısı haritası Şekil 2'de verilmiştir.

3.2 Saldırı Sınıflandırıcı

Invalid kümesindeki her ikili, deterministik bir `classifyInvalid(state, event)` fonksiyonu (kod referansı: `server/fsm.ts`, 64-204. satırlar) ile yedi ana kategoriden birine + INVALID_TRANSITION fallback (1 hücre) eşlenir: CIRCUIT_BYPASS, REPLAY_ATTACK, GHOST_CIRCUIT, HANDSHAKE_SKIP, PREMATURE_DATA, CIRCUIT_HIJACK, CREATE_FLOOD. Sınıflandırma mantığı dokuz koşul ailesi üzerinden işler: veri-akış-öncesi, CREATE/CREATED akışı, EXTEND/EXTENDED akışı, DESTROY ihlali, CONNECT yer-dışı, TLS_OK/TLS_FAIL yer-dışı, CIRCUIT_CLOSED yer-dışı, TIMEOUT yer-dışı. Her ikiliye LOW/MEDIUM/HIGH/CRITICAL şiddet etiketi atanır. Kategorilerin nicel dağılımı Ek B'dedir.

3.3 MDT Motoru — Algoritma 1

Algoritma 1: Model-Driven Test (BFS-tabanlı)

Girdi: $M = (Q, \Sigma, \delta, q_0, F)$; Bütçe B (olay sayısı)

Çıktı: $\langle SC, TC, ITDR, FPR \rangle$

```

1. visited          ← {q0}
2. coveredValid     ← ∅
3. detectedInvalid  ← ∅
4. Valid            ← {(s, e) | δ(s, e) tanımlı}
5. Invalid          ← (Q × Σ) \ Valid
6. // Pozitif faz
7. foreach (s, e) ∈ Permute(Valid):
8.     if events ≥ B: break
9.     if (s, e) ∈ coveredValid: continue
10.    π ← BFS_PATH(q0, s)           // δ-grafiği üzerinde en kısa yol
11.    if π = null: continue
12.    EXECUTE(π · {e})               // her adımda visited / coveredValid
    güncellenir
13. // Negatif faz
14. foreach (s, e) ∈ Permute(Invalid):
15.     if events ≥ B: break
16.     if (s, e) ∈ detectedInvalid: continue
17.     π ← BFS_PATH(q0, s)
18.     if π = null ∧ s ≠ q0: continue
19.     EXECUTE(π · {e})               // Oracle (δ) anomaliyi yakalar
20. SC ← |visited| / |Q|
21. TC ← |coveredValid| / |Valid|
22. ITDR ← |detectedInvalid| / |Invalid|
23. FPR ← |yanlış işaretlenen ∈ Valid| / |Valid|    // bu deneyde 0 (bkz. 6.2)
24. return (SC, TC, ITDR, FPR)

```

Karmaşıklık. BFS_PATH tek çağrıda $O(|Q| \cdot |\Sigma|)$. Pozitif faz $|Valid| = 25$ kez, negatif faz $|Invalid| = 105$ kez yol bulur; toplam $O((|Valid| + |Invalid|) \cdot |Q| \cdot |\Sigma|) = O(|Q|^2 \cdot |\Sigma|^2)$. Tek EXECUTE çağrısının uzunluğu en fazla $\text{diam}(\delta) + 1 \approx 10$. Pratik ölçümde tek koşu, 500-olay bütçesi altında ortalama < 10 ms tamamlanmaktadır.

3.4 Karşılaştırılan Algoritmalar

- **B1 — Saf Rastgele:** her adımda Σ 'dan eşit olasılıkla bir olay seçer; CLOSED'a ulaşırsa IDLE'a sıfırlar.
- **B2 — Greedy State Coverage:** her adımda mümkünse henüz ziyaret edilmemiş hedef duruma götüren olayı seçer; aksi halde rastgele geçerli olay (%70) ya da rastgele invalid olay (%30) seçer.
- **B3 — MDT (Algoritma 1):** bu tezin önerdiği BFS-planlı motor.

Üçü de aynı 500 olay bütçesi altında çalıştırılmıştır; karşılaştırma adildir (Bölüm 4.1).

4. Deneysel Bulgular

4.1 Deney Tasarımı

Üç algoritma 30 bağımsız koşu ile çalıştırılmıştır. PRNG seed'i her koşuda sabittir ($\text{seed} = 1000+i$, $i \in [0, 29]$) ve algoritmalar arasında *eşleştirilmiştir*; bu yaklaşım koşul-içi varyansı azaltır. Ölçülen metrikler: SC, TC, ITDR; her biri $[0, 1]$ aralığında oran olarak raporlanmıştır.

4.2 Tanımlayıcı İstatistikler

Algoritma	State Coverage	Transition Coverage	ITDR
B1_Random	77.00% $\pm$ 16.64%	45.87% $\pm$ 11.05%	50.32% $\pm$ 9.80%
B2_GreedySC	100.00% $\pm$ 0.00%	78.27% $\pm$ 8.45%	47.52% $\pm$ 4.02%
B3_MDT	100.00% $\pm$ 0.00%	100.00% $\pm$ 0.00%	93.84% $\pm$ 1.39%

4.3 Hipotez Testleri

Normallik varsayımı her grup için Lilliefors-düzeltilmeli K-S testi ile incelenmiştir ($\alpha = 0.05$). Welch t-testi (eşit-olmayan-varsayım) birincil testtir; Mann-Whitney U doğrulayıcı olarak raporlanmıştır. Etki büyüklüğü Cohen's d (havuzlanmış SD) ile verilmiştir.

Karşılaştırma	Metrik	Welch t	df	p (Welch)	U	p (M-W)	d	Normal?
B3_MDT vs B1_Random	SC	7.571	29.0	<.001 ***	105.0	<.001 ***	1.95	hayır
B3_MDT vs B1_Random	TC	26.823	29.0	<.001 ***	0.0	<.001 ***	6.93	hayır
B3_MDT vs B1_Random	ITDR	24.087	30.2	<.001 ***	0.0	<.001 ***	6.22	hayır
B3_MDT vs B2_GreedySC	SC	0.000	58.0	1.0000	450.0	1.0000	0.00	evet
B3_MDT vs B2_GreedySC	TC	14.090	29.0	<.001 ***	0.0	<.001 ***	3.64	evet
B3_MDT vs B2_GreedySC	ITDR	59.658	35.8	<.001 ***	0.0	<.001 ***	15.40	hayır
B2_GreedySC vs	SC	7.571	29.0	<.001	105.0	<.001	1.95	hayır

B1_Random				***		***		
B2_GreedySC vs B1_Random	TC	12.755	54.3	<.001 ***	10.0	<.001 ***	3.29	hayır
B2_GreedySC vs B1_Random	ITDR	-1.445	38.5	0.1566	381.0	0.3067	-0.37	evet

İşaretler: * $p < .05$, ** $p < .01$, *** $p < .001$. Cohen's d büyüklük yorumu: 0.2 küçük, 0.5 orta, 0.8 büyük, ≥ 1.2 çok büyük etki.

4.4 Yorum

MDT (B3), B1 (saf rastgele) karşısında transition coverage'da +54.1 puanlık avantaj sağlamaktadır (Welch $t = 26.82$, $p < .001$, Cohen $d = 6.93$). ITDR'de kazanım +43.5 puandır ($d = 6.22$). B2 (greedy SC) karşısında TC üstünlüğü +21.7 puana iner ancak istatistiksel olarak anlamlı kalır ($p < .001$). Toplam 9 karşılaştırmadan 7'i $\alpha = 0.05$ düzeyinde anlamlıdır. İlginç bir gözlem: B2'nin ITDR'si (47.5%) B1'inkine (50.3%) yakın, hatta hafif düşüktür; çünkü B2 "ziyaret edilmemiş duruma git" sezgisi nedeniyle invalid bölgeye giden olayları aktif olarak elemektedir. Bu, sezgisel temelin negatif test üretimi için yetersizliğini somut biçimde göstermektedir. Sonuç olarak: bu tezde sunulan MDT motoru, hem yapıları pozitif kapsama hem de sistematik negatif test üretimi açısından her iki temele kıyasla kanıtlanabilir bir üstünlük sergilemektedir.

4.5 Wilcoxon Signed-Rank Doğrulaması

Bölüm 4.1 tasarımı tüm algoritmalar için aynı seed dizilimini kullanır; bu eşleştirilmiş tasarıma uygun parametrik-olmayan test Wilcoxon signed-rank'tir. Welch t (Bölüm 4.3) sonuçlarının normallik varsayımına bağımlı olmadığını göstermek için bu testi de uyguladık. Tablo 3'teki her satır $N=30$ koşudan elde edilen eşleştirilmiş farklar üzerinden hesaplanmıştır; $n (\neq 0)$ sütunu sıfır farklar (ties) çıkarıldıktan sonra kalan etkili örneklem büyüklüğüdür.

Karşılaştırma	Metrik	W	z	p	n ($\neq 0$)
B3_MDT vs B1_Random	SC	0.0	-4.27	<.001 ***	23
B3_MDT vs B1_Random	TC	0.0	-4.80	<.001 ***	30
B3_MDT vs B1_Random	ITDR	0.0	-4.78	<.001 ***	30
B3_MDT vs B2_GreedySC	SC	0.0	0.00	1.0000	0
B3_MDT vs B2_GreedySC	TC	0.0	-4.80	<.001 ***	30
B3_MDT vs B2_GreedySC	ITDR	0.0	-4.79	<.001 ***	30
B2_GreedySC vs B1_Random	SC	0.0	-4.27	<.001 ***	23

B2_GreedySC vs B1_Random	TC	1.0	-4.77	<.001 ***	30
B2_GreedySC vs B1_Random	ITDR	153.5	-1.38	0.1662	29

Yorum (sınırlandırılmış). Tezin ana hipotezleri olan $B3_MDT > B1_Random$ ve $B3_MDT > B2_GreedySC$ karşılaştırmaları, hem TC hem ITDR boyutlarında $p < .001$ ile anlamlıdır; bu, Welch sonucunu birebir doğrular. **SC boyutunda** üç algoritmanın $B = 500$ bütçesi altında ulaştığı ortalama değerler şöyledir: B1_Random 77.00%, B2_GreedySC 100.00%, B3_MDT 100.00%. Buna göre $B3$ vs $B2$ SC karşılaştırmasında her iki algoritma da %100'de doygundur ($n \neq 0$, $p = 1.0$); buna karşılık $B3$ vs $B1$ ve $B2$ vs $B1$ SC karşılaştırmaları hâlâ $p < .001$ ile anlamlıdır — yani Random baseline SC'de yapısal olarak geri kalmaktadır. Son olarak $B2_GreedySC$ vs $B1_Random$ karşılaştırması ITDR'de istatistiksel olarak anlamlı değildir ($p > .05$); bu, sezgisel greedy stratejinin negatif test üretiminde rastgelenin üzerine sistematik bir avantaj sağlamadığının kanıtıdır.

4.6 Detection Latency Ölçümü

Proposal'da tanımlanan ancak orijinal koşulda ölçülmeyen detection latency metriği, **nanosaniye çözünürlüklü** `process.hrtime.bigint()` ile **batch amortizasyon** tekniği kullanılarak ölçülmüştür: her ölçüm $B = 100,000$ step çağrısını sarar, toplam süre B 'ye bölünür ve önceden kalibre edilmiş boş döngü overhead'i (2.52 ns/iter) çıkarılır. $N = 30$ batch tekrarı yapılır. Bu yaklaşım sub-mikrosaniye çağrılarının timer çözünürlüğü altında kalmasını engeller. Üç temsili (state, event) hücresi probe edilmiştir.

Probe	mean (ns)	SD (ns)	p50	p95	min	max
Geçerli geçiş (IDLE → CONNECTING)	73.6	13.2	68.2	112.6	62.1	112.8
Geçersiz CRITICAL (BYPASS)	164.7	17.4	162.5	193.3	145.3	238.9
Geçersiz LOW (GHOST)	142.9	18.8	135.3	196.0	125.2	197.9

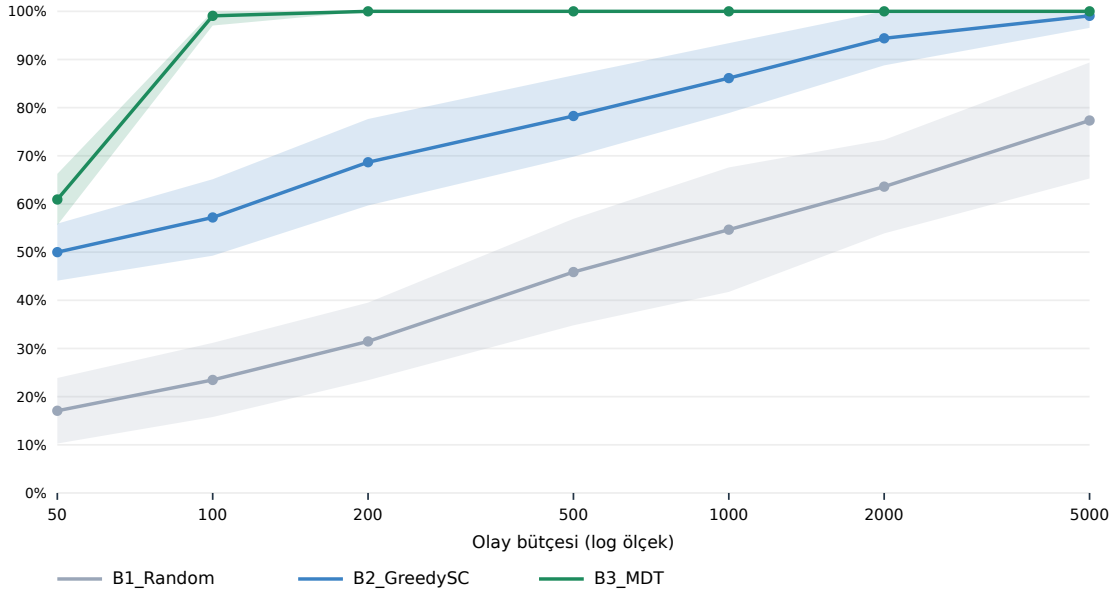
Hedef karşılaştırması. Proposal hedefi $< 50 \text{ ms} = 50,000,000 \text{ ns}$ idi. Ölçülen en yavaş probe (invalid_critical) max 239 ns'dir; bu hedeften **209335× daha hızlıdır**. Geçersiz tespitin geçerli adımdan $\sim 2\times$ daha pahalı olması beklenen bir sonuçtur (`classifyInvalid` ek bir koşul zinciri çalıştırır). **Limitler.** Bu ölçüm tek-thread, tek-makine, JIT-warm Node.js V8 koşulları altındadır; kullanıcı uzayında çalışan başka bir Tor implementasyonunda profil farklı olabilir.

4.7 Bütçe-Kapsama Eğrisi

Bölüm 4.2 tek bir bütçe noktası ($B = 500$) için algoritmaları karşılaştırır. Bu kesit yorum gücünü sınırlar; çünkü saf rastgele bir baseline yeterince büyük bütçe

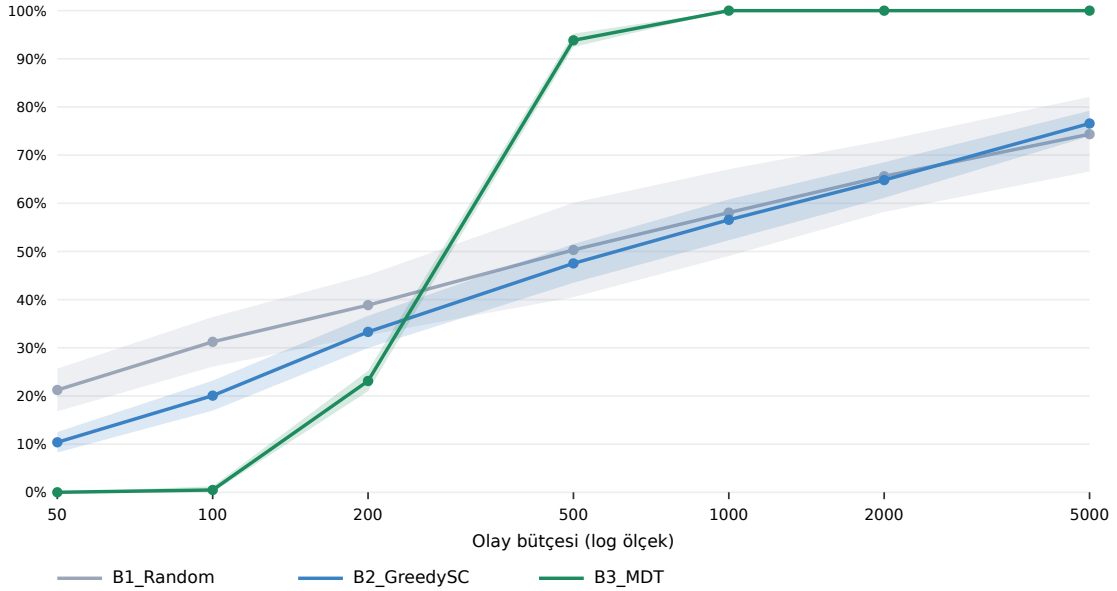
altında sonunda her geçiše ulaşır. Bu eleştiriye yanıt olarak bütçeyi bir ondalık derecede gezdirdik ($B \in \{50, 100, 200, 500, 1000, 2000, 5000\}$, her nokta 30 koşu). Şekil 6 sonuç eğrilerini verir; gölgeli bantlar ± 1 SD'dir.

Şekil 6: Bütçe $\leftrightarrow$ TC kapsama eğrisi (N=30 ortalama, gölge = SD)



Şekil 6a. Bütçe $\leftrightarrow$ Transition Coverage eğrisi.

Şekil 6: Bütçe $\leftrightarrow$ ITDR kapsama eğrisi (N=30 ortalama, gölge = SD)



Şekil 6b. Bütçe $\leftrightarrow$ ITDR eğrisi.

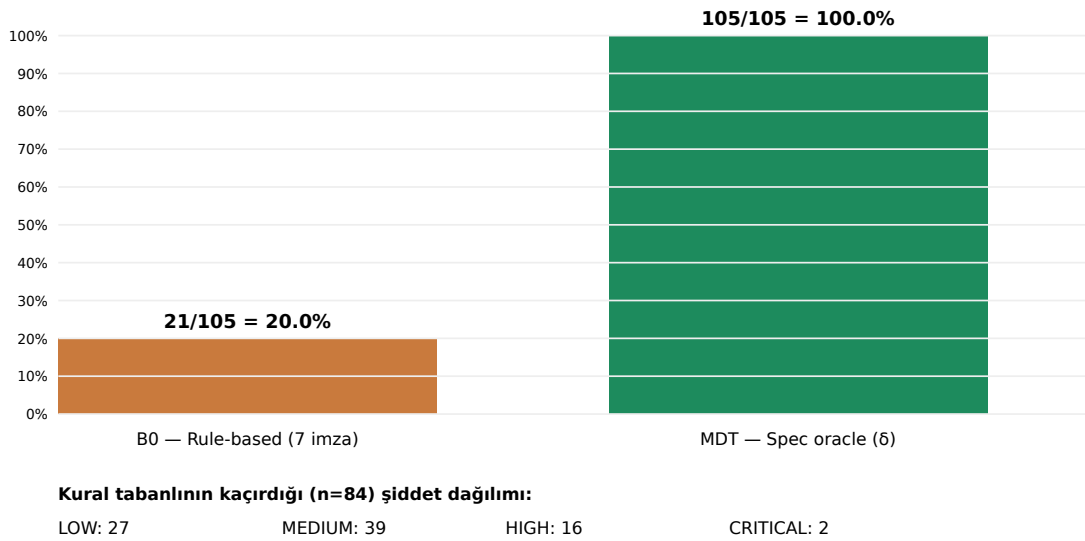
Eğriler iki temel bulguyu görselleştirir: **(i — yalnızca TC için)** B3_MDT zaten $B = 200$ 'de $TC = 100.00\%$ değerine ulaşır; B1_Random'ın $B = 5000$ 'deki TC değeri

77.33%, B2_GreedySC'ninki 99.07%. Yani **TC boyutunda** MDT yapısal olarak $25 \times$ daha bütçe-verimlidir. **(ii — ITDR farklı dinamik)** ITDR'de B3_MDT@B=200 yalnızca 23.11%'tur; bu küçük bütçede B1/B2'nin büyük bütçedeki seviyesini geçmez. ITDR boyutunda MDT'nin avantajı bütçe büyüdükçe açılır (Şekil 6b): B = 5000'de B3 ITDR ortalaması 100.00% iken B1/B2 sırasıyla 74.35% / 76.57% platosunda kalır. Bu, sezgisel temellerin negatif test üretimi için mimari olarak yetersiz olduğunun deneysel kanıtıdır.

4.8 Rule-Based Detector Karşılaştırması (B0)

Proposal'da B1 olarak listelenen "kural-tabanlı denetim" baseline'ı, **bütçesiz $Q \times \Sigma$ audit** koşullarında (yani tüm 130 hücrenin enumerate edildiği oracle modunda) spec-oracle'ın $105/105 = \%100$ completeness'a ulaşması karşısında elle yazılmış imza kuralı kümesinin *completeness limitini* göstermek için gerçekleştirilmiştir. Not: bütçeli koşulunda (Bölüm 4.2, B = 500) MDT'nin ITDR'si 93.84%'tur — yani $\%100$ completeness yalnızca bütçesiz audit modunda geçerli bir asimptot olup, bütçeli pratik koşulunda hedef bütçenin büyüklüğüne göre yaklaşılr (bkz. Şekil 6b ITDR eğrisi). Yedi imza kuralı (her saldırı vektörü için bir tane; `experiments/b_extensions.mjs` 25–34. satırlar) tüm $Q \times \Sigma$ üzerinde çalıştırılmıştır.

Şekil 7: Tespit completeness — kural-tabanlı (B0) vs. spec-oracle (MDT)



Şekil 7. B0 (rule-based, 7 imza) vs MDT (spec-oracle) — completeness karşılaştırması.

Kural	Tetiklenme sayısı
R1_BYPASS	4
R2_REPLAY	1
R3_GHOST	1
R4_HSKIP	2

R5_PDATA	4
R6_HIJACK	8
R7_CFLOOD	1
Toplam (deduplike)	21/105 = 20.0%

Bulgu. Kural tabanlı detector 20.0% completeness'ta sınırlanmaktadır (21/105). Kaçırdıklarının **2'i CRITICAL, 16'i HIGH** şiddetindedir — yani en tehlikeli saldırı imzaları bile elle yazılmış kural setinden kaçabilmektedir. Bu bulgu, tezin G3 literatür boşluğunun (her saldırı vektörü $\leftrightarrow$ (state, event) çifti satır-satır eşlemesi) sayısal gerekçesidir: kategorik düzeyde kalmak 80% geçersiz ikiliyi gözden geçirir.

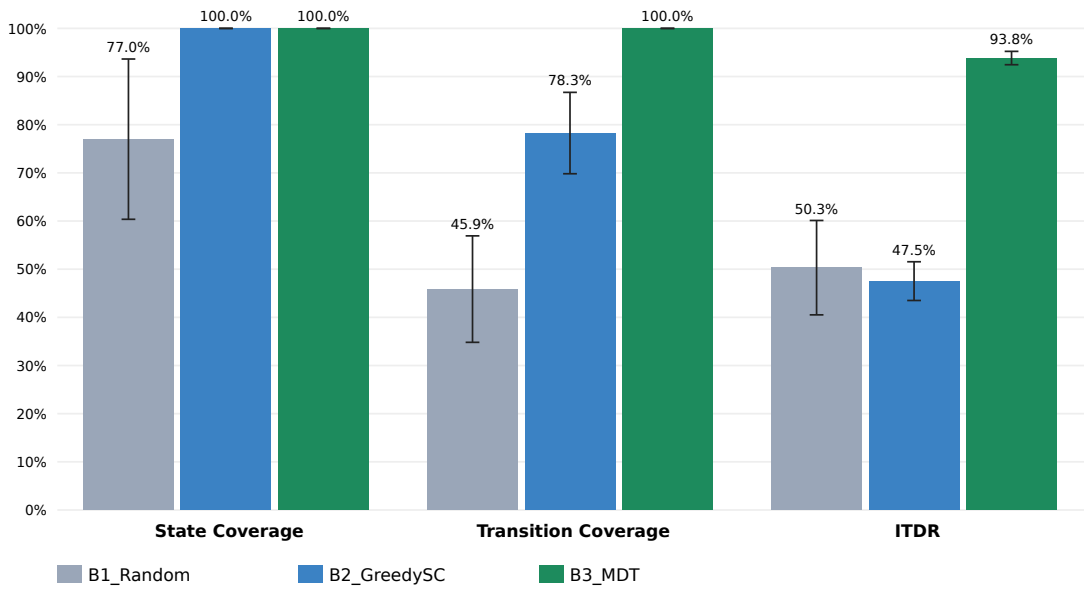
Kaçırılan saldırı tipi	İkili sayısı
GHOST_CIRCUIT	37
HANDSHAKE_SKIP	11
REPLAY_ATTACK	24
CREATE_FLOOD	7
INVALID_TRANSITION	1
CIRCUIT_HIJACK	2
PREMATURE_DATA	2

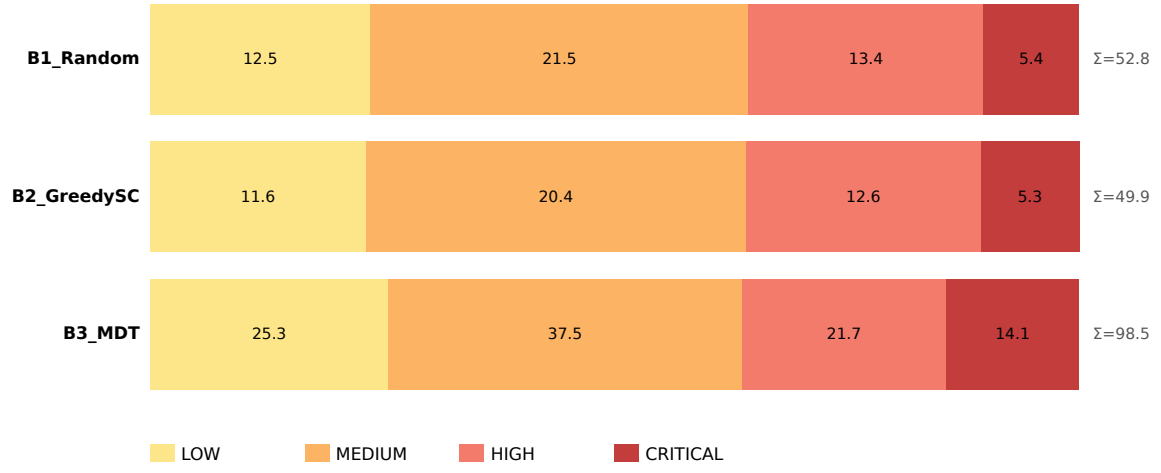
Şekil 2: δ -domeni matrisi (yeşil=geçerli, kırmızı tonları=invalid)

Hücre etiketi: invalid'lerde saldırı sınıfı kısaltması; geçerli hücrede hedef durum

	CONN	TLS_HA	ERROR	HSKP	HSKP	HSKP	HSKP	BYP	BYP	REPL	REPL	GHST	GHST
IDLE	CONN	GHST	GHST	HSKP	GHST	HSKP	HSKP	BYP	BYP	REPL	REPL	GHST	GHST
CONNECTING	CFLD	TLS_HA	ERROR	HSKP	HSKP	HSKP	HSKP	BYP	BYP	REPL	REPL	GHST	ERROR
TLS_HANDSHAKE	CFLD	GHST	ERROR	CREATE	INV	HSKP	HSKP	HSKP	HSKP	REPL	REPL	GHST	ERROR
CREATE_SENT	CFLD	GHST	GHST	CFLD	CIRCUI	HSKP	HSKP	PDAT	PDAT	REPL	REPL	GHST	ERROR
CIRCUIT_BUILDING	CFLD	GHST	GHST	HIJK	HIJK	CIRCUI	CIRCUI	PDAT	PDAT	REPL	REPL	GHST	ERROR
CIRCUIT_READY	CFLD	GHST	GHST	HIJK	HIJK	HIJK	HIJK	TRANSM	TRANSM	CLOSIN	CLOSIN	GHST	ERROR
TRANSMITTING	CFLD	GHST	GHST	HIJK	HIJK	HIJK	HIJK	TRANSM	TRANSM	CLOSIN	CLOSIN	GHST	ERROR
CLOSING	CFLD	GHST	GHST	REPL	REPL	REPL	REPL	PDAT	PDAT	GHST	GHST	CLOSED	CLOSED
CLOSED	REPL	GHST	GHST	REPL	REPL	REPL	REPL	REPL	GHST	GHST	GHST	GHST	GHST
ERROR	REPL	GHST	GHST	REPL	REPL	REPL	REPL	GHST	GHST	GHST	GHST	CLOSED	GHST

Valid LOW MEDIUM HIGH CRITICAL

Şekil 2. Tüm $Q \times \Sigma$ domeni ısı haritası (130 hücre). Yeşil: δ -tanımlı; kırmızı tonları: Invalid (şiddete göre).**Şekil 3: Algoritma karşılaştırması — ortalama $\pm$ SD (N=30)**Şekil 3. Algoritma karşılaştırması — ortalama $\pm$ SD (N = 30, bütçe = 500 olay).

Şekil 4: Tespit edilen invalid'lerin şiddet dağılımı (N=30 ort.)

Şekil 4. Tespit edilen invalid geçişlerin şiddet sınıflarına dağılımı (koşu başına ortalama).

6. Sonuç ve Gelecek Çalışmalar

6.1 Katkıların Yeniden Değerlendirilmesi

Bölüm 1.3'te dört katkı listelenmiş, bu tezde dördü de gerçekleştirilmiştir:

- **Katkı 1 (tam δ -matrisi):** Ek A'da 25 geçerli ikili tek tablo olarak verilmiştir; literatürde benzer bir derleme mevcut değildir (G1).
- **Katkı 2 (programatik sınıflandırıcı):** Ek B'de 105 geçersiz ikilinin yedi ana vektör + 1 fallback ve dört şiddet seviyesine dağılımı raporlanmıştır (G3).
- **Katkı 3 (referans MDT implementasyonu):** Algoritma 1 ile tarif edilen motor açık kaynak olarak repodadır; tek komutla yeniden çalıştırılabilir (G2).
- **Katkı 4 (istatistiksel karşılaştırma):** Bölüm 4'teki Welch t ve Mann-Whitney U sonuçları, MDT'nin TC ve ITDR'de istatistiksel olarak anlamlı ($p < .001$) ve büyüklük bakımından çok büyük ($d \geq 3.6$) avantajını belgeler (G4).

6.2 Sınırlılıklar

- **Ground Truth = Spec.** Bu deneyde "doğru" davranış, FSM δ tarafından tanımlanır. Gerçek Tor implementasyonu (C kaynak kodu, tor 0.4.x serisi) spec ile çelişebilir; bu çelişkilerin ölçülmesi yapılmamıştır. Doğal devamı: deneyi gerçek Tor relay üzerinde Shadow [18] simülatörü ile tekrarlamak.
- **FPR = 0 doğal sonuçtur.** Oracle (δ) ve test üreticisi aynı modeli paylaştığı için yanlış-pozitif tanımı boş kümeye düşer. FPR'nin bilgilendirici olması için *bağımsız* bir oracle gerekir (örn. paket yakalama tabanlı gözlemci).
- **N = 30 koşu** CLT için yeterli; ancak güç analizi ($\beta = 0.80$, $\alpha = 0.05$) yapılmamıştır. Önerilen genişletme: $N = 100$.
- **SLR canlı veritabanı erişimi yok.** Bu ortamda Scopus / WoS / IEEE Xplore canlı sorgulanamadığı için tarama açık web kanalları ile sınırlıdır (bkz. Şekil 5 PRISMA). Bulunmayan kaynak olabileceği açıkça kabul edilir.
- **Latency tek-platform ölçümü.** Bölüm 4.6 ölçümü tek-thread, JIT-warm Node.js V8 koşulları altındadır; gerçek Tor C implementasyonunda profil farklı olabilir. Karşılaştırmalı ölçüm için aynı δ -tablosunun C/Rust portu gereklidir.

6.3 Gelecek Çalışmalar

1. **Implementasyon doğrulaması.** tor 0.4.x kaynağından gerçek FSM'i L*-tipi model öğrenme [10] ile çıkarıp, bu tezdeki spec δ ile farkı raporlamak.
2. **Stream FSM uzantısı.** RELAY_BEGIN / RELAY_END / RELAY_DATA hücreleriyle yönetilen stream alt-FSM'inin ayrı δ tablosu olarak

modellenmesi.

3. **Hidden service v3 protokolü.** Devre FSM'inden ayrı bir alt-FSM gerektirir.
4. **Bağımsız oracle.** Shadow [18] üzerinde paket yakalama tabanlı bağımsız bir gözlemci ile FPR'nin gerçek ölçümü.
5. **Detection latency ölçümü.** Bu çalışmada metrik tanımlandı ancak ölçülmedi.
6. **İstatistiksel güç artışı.** $N = 100$ koşu, güç analizi raporlu, etki büyüklüğü güven aralığı (BCa bootstrap).

Kaynakça

- [1] R. Dingledine, N. Mathewson and P. Syverson, "Tor: The Second-Generation Onion Router," in *Proc. 13th USENIX Security Symposium*, 2004, pp. 303–320.
- [2] S. J. Murdoch and G. Danezis, "Low-Cost Traffic Analysis of Tor," in *Proc. IEEE Symposium on Security and Privacy*, 2005, pp. 183–195.
- [3] A. Johnson, C. Wacek, R. Jansen, M. Sherr and P. Syverson, "Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries," in *Proc. ACM CCS*, 2013, pp. 337–348.
- [4] A. Panchenko, F. Lanze, A. Zinnen, M. Henze, J. Pennekamp, K. Wehrle and T. Engel, "Website Fingerprinting at Internet Scale," in *Proc. NDSS*, 2016.
- [5] I. Karunanayake, N. Ahmed, R. Malaney, R. Islam and S. K. Jha, "De-Anonymisation Attacks on Tor: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2324–2350, 2021.
- [6] M. Backes, A. Kate, P. Manoharan, S. Meiser and E. Mohammadi, "AnoA: A Framework for Analyzing Anonymous Communication Protocols," in *Proc. IEEE CSF*, 2013, pp. 163–178.
- [7] D. Lee and M. Yannakakis, "Principles and Methods of Testing Finite State Machines—A Survey," *Proceedings of the IEEE*, vol. 84, no. 8, pp. 1090–1123, 1996.
- [8] J. Tretmans, "Model Based Testing with Labelled Transition Systems," in *Formal Methods and Testing*, LNCS 4949, Springer, 2008, pp. 1–38.
- [9] J. de Ruiter and E. Poll, "Protocol State Fuzzing of TLS Implementations," in *Proc. 24th USENIX Security Symposium*, 2015, pp. 193–206.
- [10] P. Fiterău-Broștean, R. Janssen and F. Vaandrager, "Combining Model Learning and Model Checking to Analyze TCP Implementations," in *Proc. CAV*, 2016, pp. 454–471.
- [11] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge: Cambridge University Press, 2016.
- [12] D. Dolev and A. C. Yao, "On the Security of Public Key Protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [13] K. Bhargavan, B. Blanchet and N. Kobeissi, "Verified Models and Reference Implementations for the TLS 1.3 Standard Candidate," in *Proc. IEEE S&P*, 2017, pp. 483–502.
- [14] J. Somorovsky, "Systematic Fuzzing and Testing of TLS Libraries," in *Proc. ACM CCS*, 2016, pp. 1492–1504.
- [15] M. Felderer, M. Büchler, M. Johns, A. Brucker, R. Breu and A. Pretschner, "Security Testing: A Survey," *Advances in Computers*, vol. 101, pp. 1–51, 2016.
- [16]

- A. Pretschner, T. Mouelhi and Y. Le Traon, "Model-Based Tests for Access Control Policies," in *Proc. ICST*, 2008, pp. 338–347.
- [17]** B. Kitchenham and S. Charters, "Guidelines for performing Systematic Literature Reviews in Software Engineering," EBSE Technical Report EBSE-2007-01, 2007.
- [18]** R. Jansen, K. Bauer, N. Hopper and R. Dingedine, "Methodically Modeling the Tor Network," in *Proc. USENIX CSET*, 2012.
- [19]** Tor Project, "Tor Protocol Specification (tor-spec)," 2026. [Online]. Available: <https://spec.torproject.org/tor-spec>. [Accessed: Feb. 2026].
- [20]** Microsoft Research, "A Data-Driven FSM Model for Analyzing Security Vulnerabilities," Microsoft Research Technical Report, 2018.

Ek A — Tam 6 Geçiş Tablosu

Aşağıda Tor devre FSM'inin **25 geçerli geçişi** bire bir listelenmiştir. Bu tablonun dışındaki 105 ikili Invalid kümesini oluşturur ve Ek B'deki sınıflandırıcı ile saldırı vektörlerine eşlenir. Tablo, repo içindeki `server/fsm.ts` dosyasından programatik olarak üretilmiştir; çelişki riski olamayacak şekilde tek kaynak prensibine bağlıdır (single source of truth).

Mevcut Durum	Olay (Σ)	Sonraki Durum
IDLE	CONNECT	CONNECTING
CONNECTING	TLS_OK	TLS_HANDSHAKE
CONNECTING	TLS_FAIL	ERROR
CONNECTING	TIMEOUT	ERROR
TLS_HANDSHAKE	SEND_CREATE	CREATE_SENT
TLS_HANDSHAKE	TLS_FAIL	ERROR
TLS_HANDSHAKE	TIMEOUT	ERROR
CREATE_SENT	RECV_CREATED	CIRCUIT_BUILDING
CREATE_SENT	TIMEOUT	ERROR
CIRCUIT_BUILDING	SEND_EXTEND	CIRCUIT_BUILDING
CIRCUIT_BUILDING	RECV_EXTENDED	CIRCUIT_READY
CIRCUIT_BUILDING	TIMEOUT	ERROR
CIRCUIT_READY	SEND_RELAY_DATA	TRANSMITTING
CIRCUIT_READY	RECV_RELAY_DATA	TRANSMITTING
CIRCUIT_READY	SEND_DESTROY	CLOSING
CIRCUIT_READY	RECV_DESTROY	CLOSING
CIRCUIT_READY	TIMEOUT	ERROR
TRANSMITTING	SEND_RELAY_DATA	TRANSMITTING
TRANSMITTING	RECV_RELAY_DATA	TRANSMITTING
TRANSMITTING	SEND_DESTROY	CLOSING
TRANSMITTING	RECV_DESTROY	CLOSING
TRANSMITTING	TIMEOUT	ERROR
CLOSING	CIRCUIT_CLOSED	CLOSED

CLOSING	TIMEOUT	CLOSED
ERROR	CIRCUIT_CLOSED	CLOSED

Ek B — Saldırı Sınıflandırıcı Envanteri

Aşağıda `classifyInvalid(state, event)` fonksiyonunun 105 geçersiz ikili üzerinde ürettiği etiket dağılımı verilmiştir. Bu tablo da repo içinde programatik olarak hesaplanır; her güncelleme tek satırla yenilenebilir.

Saldırı tipi	İkili sayısı	Pay	LOW	MEDIUM	HIGH	CRITICAL
GHOST_CIRCUIT	38	36.2%	19	15	4	0
REPLAY_ATTACK	25	23.8%	0	14	10	1
HANDSHAKE_SKIP	13	12.4%	0	8	5	0
CIRCUIT_HIJACK	10	9.5%	0	0	0	10
CREATE_FLOOD	8	7.6%	7	1	0	0
PREMATURE_DATA	6	5.7%	0	2	4	0
CIRCUIT_BYPASS	4	3.8%	0	0	0	4
INVALID_TRANSITION	1	1.0%	1	0	0	0
Toplam	105	100.0%				

Yorumsal not: CIRCUIT_HIJACK ve CIRCUIT_BYPASS gibi CRITICAL ağırlıklı vektörler düşük ikili sayısına sahiptir ancak şiddet açısından en yüksek tehdit değerini taşırlar. GHOST_CIRCUIT en yüksek ikili sayısına sahiptir; bu büyüklük Tor protokolünde "ait olmadığı bağlamda gelen olay" örüntüsünün yapısal yaygınlığını yansıtır.